

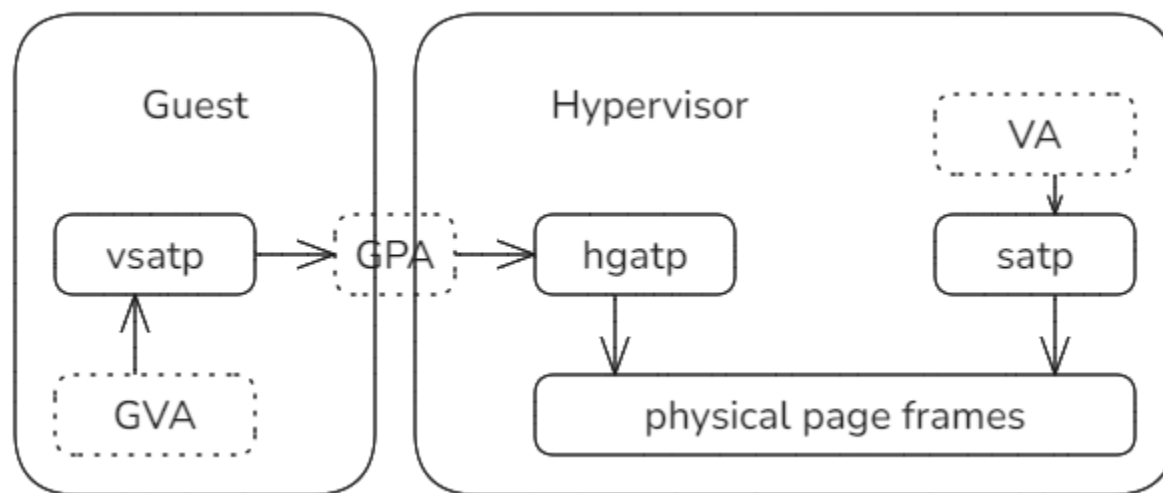
秋冬季训练营三阶段 组件化内核设计与实践(9)

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

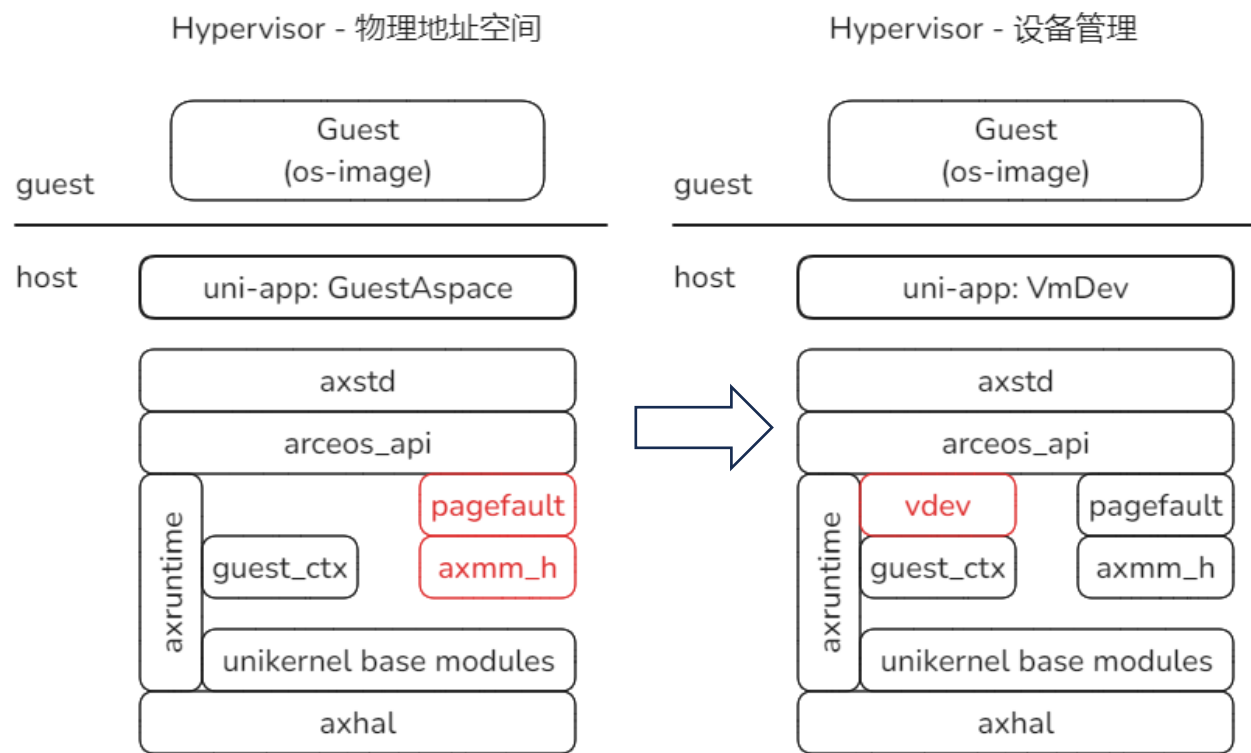
2024.11.29

两阶段地址映射的补充说明

- 在Hypervisor内部: VA -> PA
- 来自Guest则可能经历两阶段映射: GVA->GPA->HPA



H.3.0 & H.4.0: VmDev

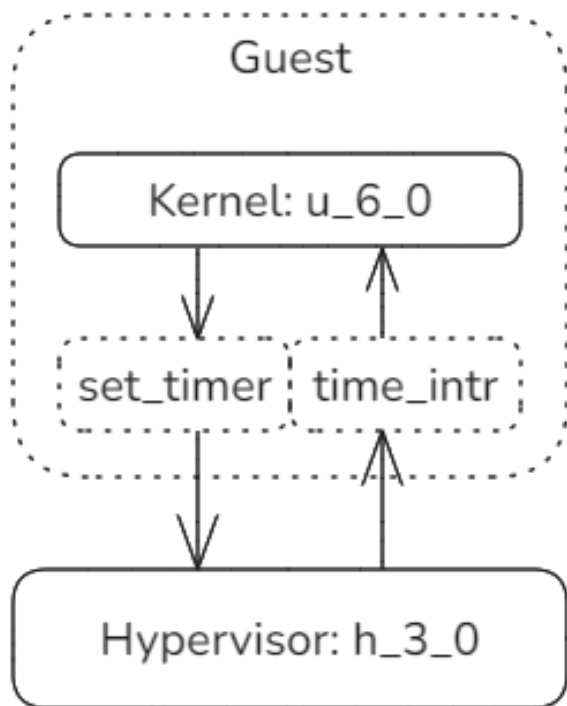


本节目标:

1. 虚拟时钟中断支持; 虚拟机外设的支持。
2. 虚拟机中运行宏内核和通过透传pflash加载应用。

H.3.0实验目标 - 虚拟机时钟中断

通过实验h_3_0学习虚拟机时钟中断的实现原理，配合实验的Guest内核是u_6_0。
u_6_0中的抢占式调度机制依赖时钟中断，它启用时钟定时器，不断的触发时钟中断。
由于当前该内核是在Guest虚拟机中运行，Hypervisor必须能够模拟物理机的中断能力。



h_3_0实验步骤:

```
make A=tour/u_6_0/  
./update_disk.sh tour/u_6_0/u_6_0_riscv64-qemu-virt.bin  
make run A=tour/h_3_0 BLK=y LOG=info
```

执行结果：在worker执行序列中夹杂着下面的日志。

```
Set timer...
```

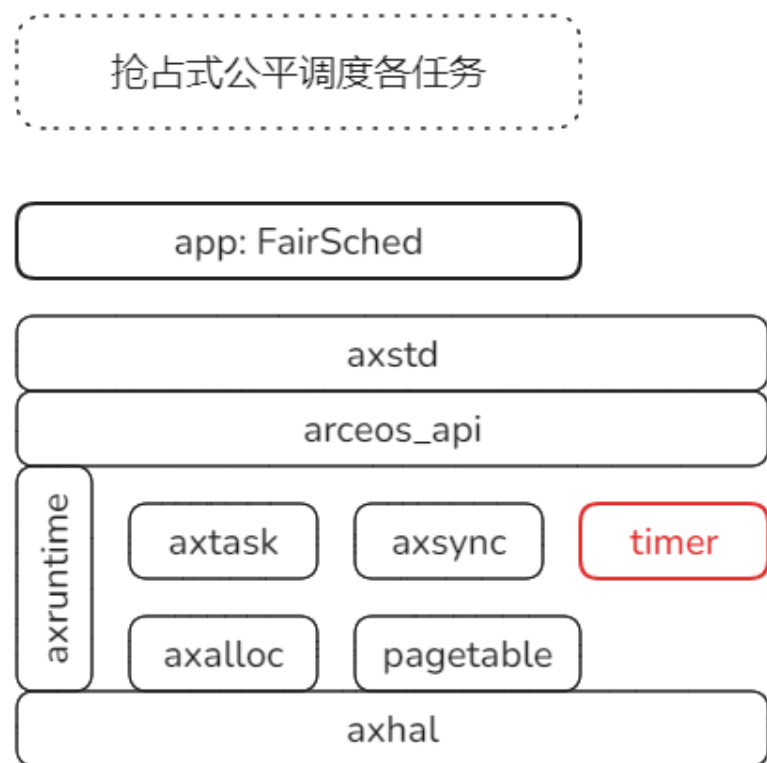
Guest内核设置定时器

```
timer irq emulation
```

时钟中断注入虚拟机

回顾实验U_6_0的内容

在u_6_0实验中，支持了抢占式调度算法CFS，该机制必须依赖于时钟中断。



通过axhal 注册**时钟中断**，定期触发
axtask::on_timer_tick



经由axtask::runqueue传递定时事件



触发特定调度器的task_tick，决定是
否标记抢占标志，并可能进一步的导
致任务队列的重排

axruntime和axhal对时钟中断的处理

modules/axruntime/src/lib.rs

```
axhal::irq::register_handler(TIMER_IRQ_NUM, || {
    update_timer();
    #[cfg(feature = "multitask")]
    axtask::on_timer_tick();
});

fn update_timer() {
    let now_ns = axhal::time::monotonic_time_nanos();
    // Safety: we have disabled preemption in IRQ handler.
    let mut deadline = unsafe { NEXT_DEADLINE.read_current_raw() };
    if now_ns >= deadline {
        deadline = now_ns + PERIODIC_INTERVAL_NANOS;
    }
    unsafe {
        NEXT_DEADLINE.write_current_raw(
            deadline + PERIODIC_INTERVAL_NANOS
        );
    }
    axhal::time::set_oneShot_timer(deadline);
}
```



在异常中断向量表中注册
对应时钟中断的响应函数
更新时钟和触发定时事件

每次响应时钟中断时
会重置下次的时钟

modules/axhal/src/platform/riscv64_qemu_virt/time.rs

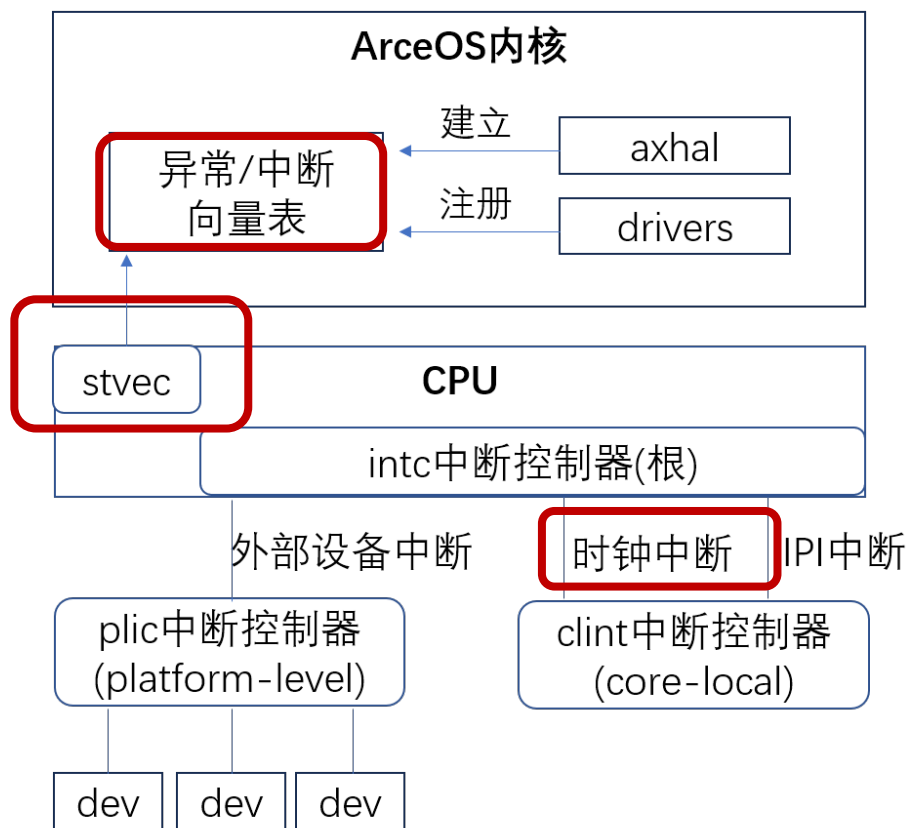
```
#[cfg(feature = "irq")]
pub fn set_oneShot_timer(deadline_ns: u64) {
    sbi_rt::set_timer(nanos_to_ticks(deadline_ns));
}
```

重置时钟的方法
通过调用SBI-Call的set_timer功能调用实现

在虚拟机中运行u_6_0面临的问题

物理环境或者qemu模拟器中，时钟中断触发时，能够正常通过stvec寄存器找到异常中断向量表，然后进入事先注册的响应函数。但是在虚拟机环境下，宿主环境下的原始路径失效了。有两种解决方案：

1. 启用Riscv AIA机制，把特定的中断委托到虚拟机Guest环境下。要求平台支持，且比较复杂。
2. 通过中断注入的方式来实现。即本实验采取的方式。



体系结构对中断注入的支持

注入机制的关键是寄存器hvip，指示在Guest环境中，哪些中断处于Pending状态。
手册第18.2.3. Hypervisor Interrupt Registers描述了hvip寄存器的作用和格式。

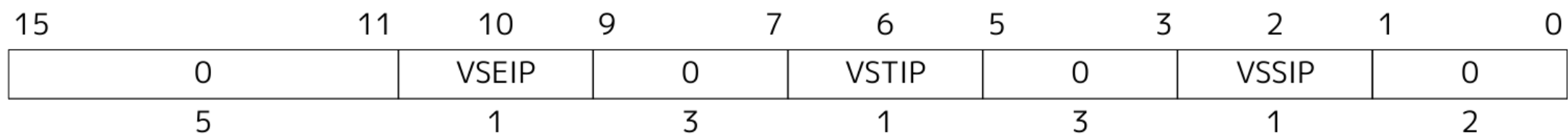


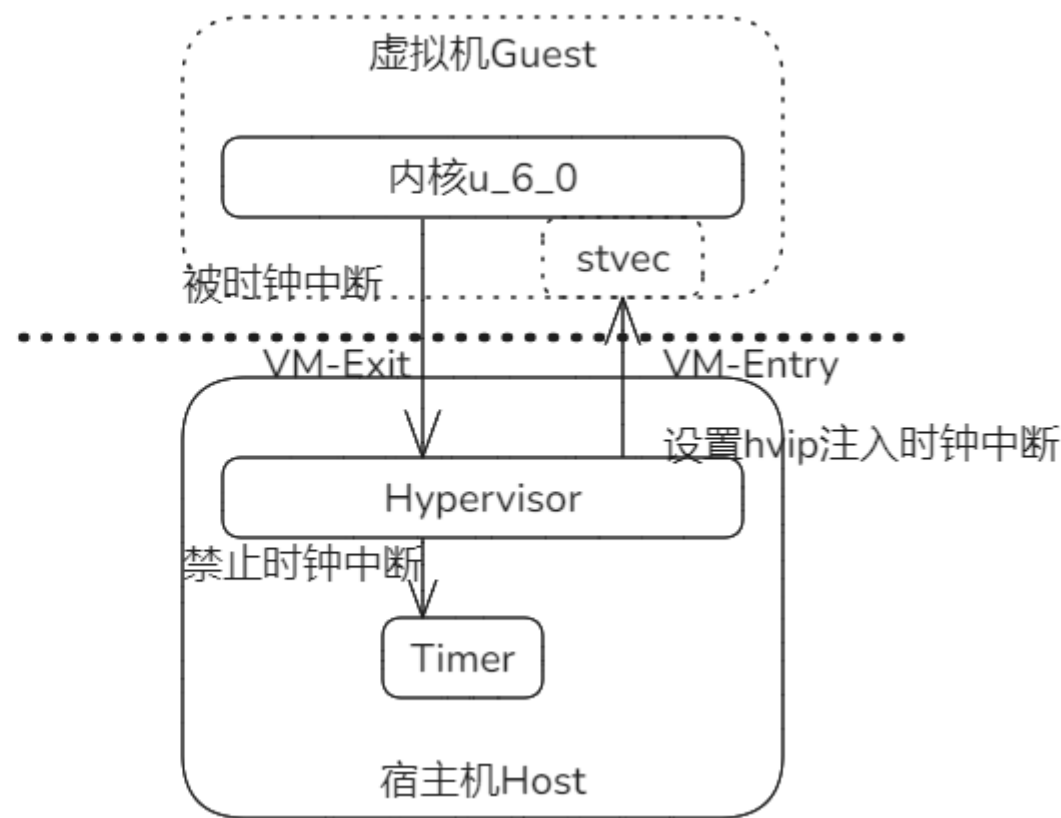
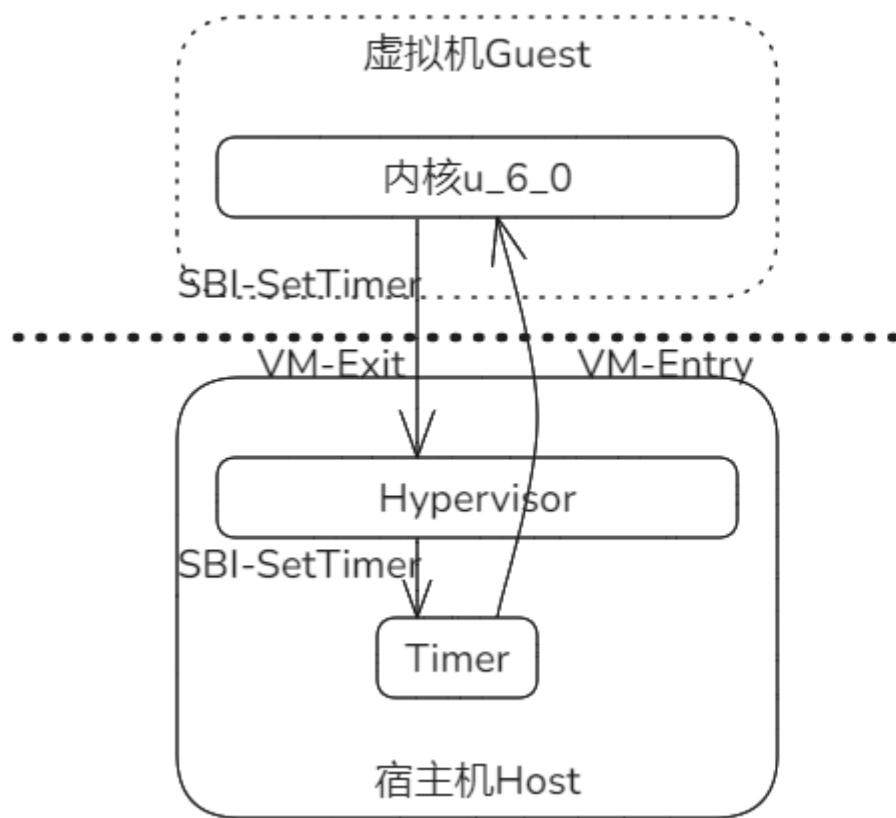
Figure 74. Standard portion (bits 15:0) of hvip.

可以分别对外部中断、时钟中断和IPI中断进行设置，1表示有中断待处理。

时钟中断设置和注入的实现

支持虚拟机时钟中断需要实现两部分的内容：

1. 响应虚拟机发出的SBI-Call功能调用SetTimer
2. 响应宿主机时钟中断导致的VM退出，注入到虚拟机内部



时钟中断设置和注入的实现

两种VM-EXIT的处理都在vmexit_handler中，实现在文件modules/riscv_vcpu/src/vcpu.rs。

响应虚拟机发出的SBI-Call功能调用SetTimer

```
SbiMessage::SetTimer(timer) => {
    info!("Set timer... ");
    sbi_rt::set_timer(timer as u64);
    // Clear guest timer interrupt
    CSR.hvip.read_and_clear_bits(
        traps::interrupt::VIRTUAL_SUPERVISOR_TIMER
    );
    // Enable host timer interrupt
    CSR.sie.read_and_set_bits(
        traps::interrupt::SUPERVISOR_TIMER
    );
}
```

- 1) 把hvip的时钟中断对应位清零，确保此时不会触发虚拟机内部的时钟中断
- 2) 启用Host宿主机的时钟中断

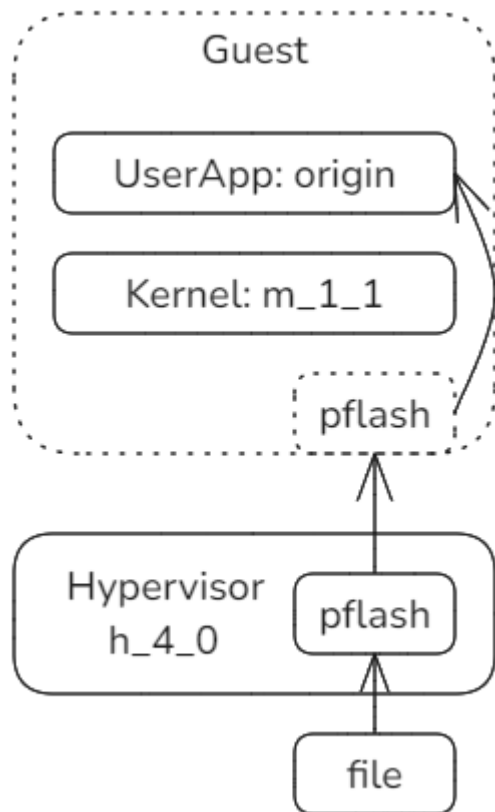
响应宿主机时钟中断导致的VM退出，注入到虚拟机内部

```
Trap::Interrupt(Interrupt::SupervisorTimer) => {
    info!("timer irq emulation");
    // Enable guest timer interrupt
    CSR.hvip.read_and_set_bits(
        traps::interrupt::VIRTUAL_SUPERVISOR_TIMER
    );
    // Clear host timer interrupt
    CSR.sie.read_and_clear_bits(
        traps::interrupt::SUPERVISOR_TIMER
    );
    Ok(AxVCpuExitReason::Nothing)
}
```

- 1) 把hvip的时钟中断对应位设置1，向虚拟机内部注入时钟中断
- 2) 暂时禁用Host宿主机的时钟中断，等待下次虚拟机请求时再启用

H.4.0实验目标 - 支持宏内核作为GuestOS

h_4_0是虚拟机中启动宏内核的实验，配合实验的Guest内核是m_1_1。
m_1_1从pflash载入第一个用户应用并启动，响应syscall后退出。
Hypervisor透传pflash给虚拟机，让m_1_1内核能够正常工作。



h_4_0实验步骤：

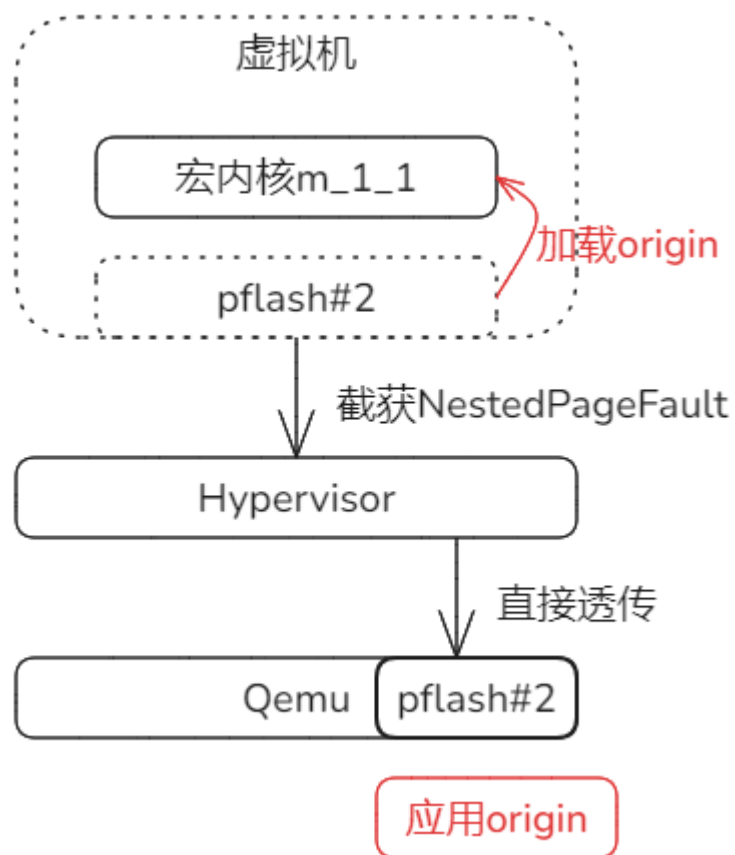
```
make A=tour/m_1_1
./update_disk.sh ./tour/m_1_1/m_1_1_riscv64-qemu-virt.bin
make run A=tour/h_4_0 BLK=y
```

执行结果：能够看到两次打印ArceOS的Logo。
前一次是Hypervisor启动，后一次是宏内核启动。

```
Enter user space: entry=0x1000, ustack=0x4000000000,
handle_syscall ...
[SYS_EXIT]: process is exiting ..
monolithic kernel exit [Some(0)] normally!
cloud@server:~/gitWork/oscamp/arceos$
```

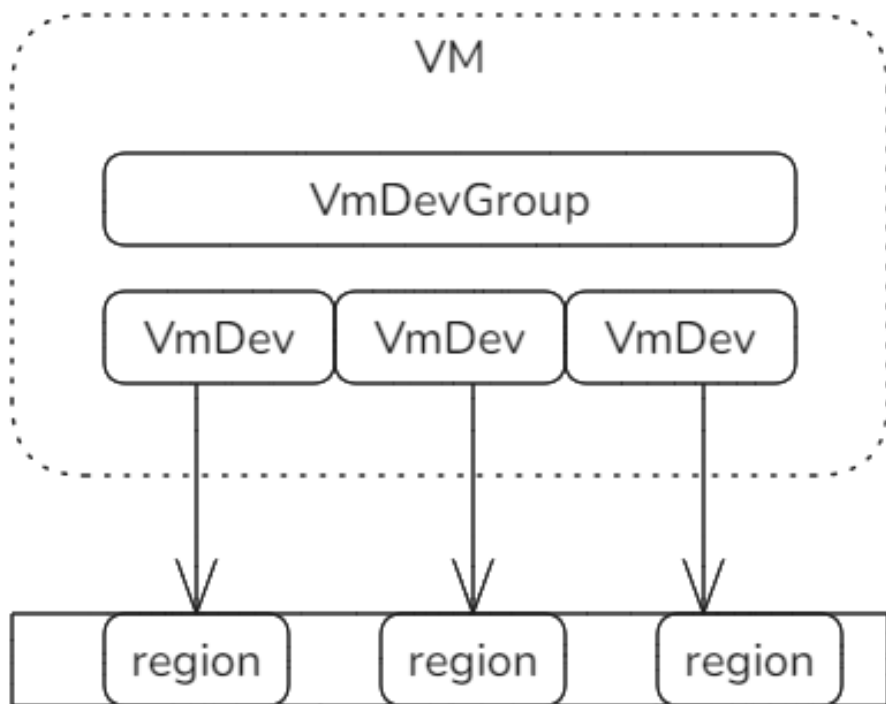
虚拟机中支持宏内核要解决的问题

1. Hypervisor需要加载宏内核的Image文件
2. 当宏内核启动后，Hypervisor支持某种块存储，让宏内核能够从中加载应用的Image文件
前者直接从disk.img中加载，后者则参照h_2_0实验的方式，通过透传pflash，让宏内核能够从中读出应用Image，加载并执行。



虚拟机对设备的管理

管理上的层次结构：虚拟机（VM），设备组VmDevGroup以及设备VmDev。
Riscv架构下，虚拟机包含的各种外设通常是通过MMIO方式访问，
因此主要用地址范围标记它们的资源。实现文件tour/h_4_0/src/vmdev.rs



AddrSpace Portion for MMIO

```
pub struct VmDevGroup {
    devices: Vec<Arc<VmDev>>
}
```

简单采用Vec管理，提供add_dev注册方法和find_dev的查找方法。

```
pub struct VmDev {
    start: VirtAddr,
    size: usize,
}
```

标记mmio区域的起止地址，提供handle_mmio()方法响应访问。

实验H_4_0的实现

相对于h_2_0，当前实验支持了以宏内核为GuestOS内核，然后建立了设备管理机制。

1. 采取注册的方式，把各种支持设备注册到设备管理组中。

```
// Register pflash device into vm.  
let mut vmdevs = VmDevGroup::new();  
vmdevs.add_dev(0x2200_0000.into(), 0x200_0000);
```

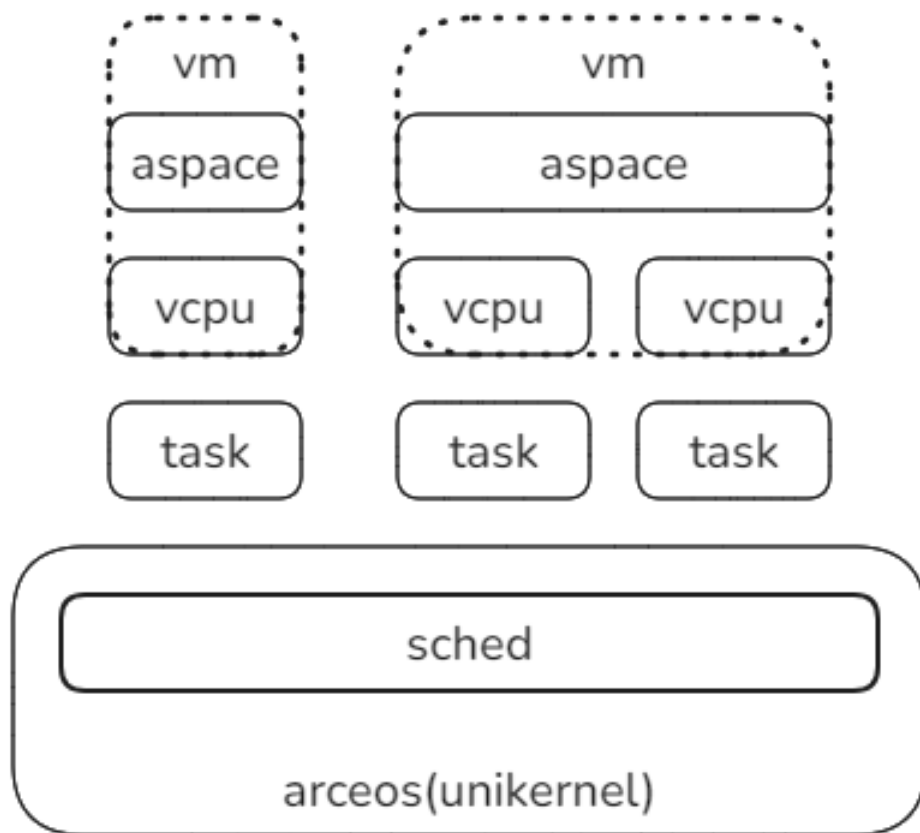
2. 发生页面异常时，基于地址查找目标设备，调用该设备的响应方法处理请求。

```
match vcpu_run(&mut arch_vcpu) {  
    Ok(exit_reason) => match exit_reason {  
        AxVCpuExitReason::Nothing => {},  
        NestedPageFault{addr, access_flags} => {  
            debug!("addr {:#x} access {:#x}", addr, access_flags);  
            if addr < PHY_MEM_START.into() {  
                // Find dev and handle mmio region.  
                let dev = vmdevs.find_dev(addr).expect("No dev.");  
                dev.handle_mmio(addr, &mut aspace).unwrap();  
            } else {  
                unimplemented!("Handle #PF for memory region.");  
            }  
        }  
    },
```

目前实验未涵盖的内容：多vCPU的支持

Hypervisor中，每个vCPU代表一个执行流，因此对应一个任务Task，可以复用ArceOS Unikernel中介绍的调度机制和算法。

一个虚拟机VM可以配置多个vCPU，但是站在调度器的角度，它只管按任务调度。vCPU关联的地址空间ASpace的根页表，有可能会随着任务切换而切换。

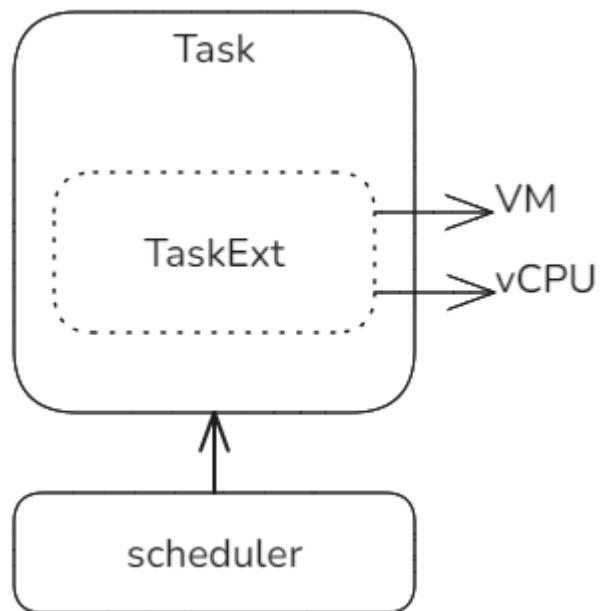


目前实验未涵盖的内容：扩展Task实现vCPU

当前实验是简化模型，没有启动任务来运行vCPU。

完整的实现需要为每个vCPU启动一个Task任务来运行。

机制上采用TaskExt来扩展。与宏内核实验中为进程/线程实现Task的方式相同。



```
use axvm::{AxVCpuRef, AxVMRef};

use crate::hal::AxVMHalImpl;

/// Task extended data for the monolithic kernel.
pub struct TaskExt {
    /// The VM.
    pub vm: AxVMRef<AxVMHalImpl>,
    /// The vCPU.
    pub vcpu: AxVCpuRef,
}

impl TaskExt {
    pub const fn new(vm: AxVMRef<AxVMHalImpl>, vcpu: AxVCpuRef) -> Self {
        Self { vm, vcpu }
    }
}

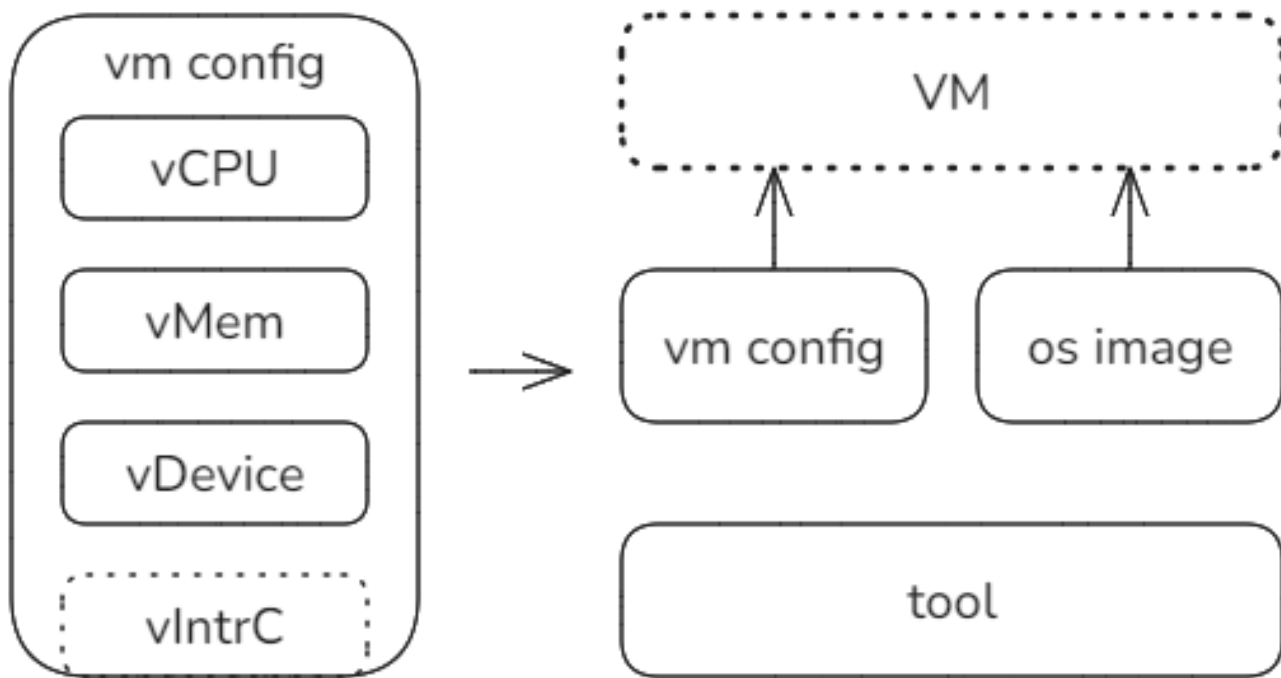
axtask::def_task_ext!(TaskExt);
```


目前实验未涵盖的内容： hypervisor的配置

当前的简化实验中，直接硬编码了配置参数。

未来可以考虑基于配置文件 + OS Image文件的方式通过专门工具构建和管理虚拟机。

形式上就类似于VMWare等软件的方式。



课后练习

目前tour下针对Hypervisor的几个实验，已经把前两周构建的部分内核作为GuestOS内核在虚拟机中成功运行，既包括Unikernel，也包括宏内核。

请各位尝试基于H_4_0运行目前实验中没有涵盖的其它Unikernel和宏内核。